

UNIVERSIDAD TECNOLÓGICA DE QUERÉTARO

Nombre de los integrantes:

Elian Eduardo Ramírez Ramírez

Grupo:

IDGS17

Nombre de la actividad:

Práctica 4. Herramientas para Administración de Logs. Parte 2

Nombre de la materia:

Seguridad en el Desarrollo de Aplicaciones IDGS17

SEGG-U2-P4H-2

Integración y Enriquecimiento de Registros con Promtail y Loki

Objetivo

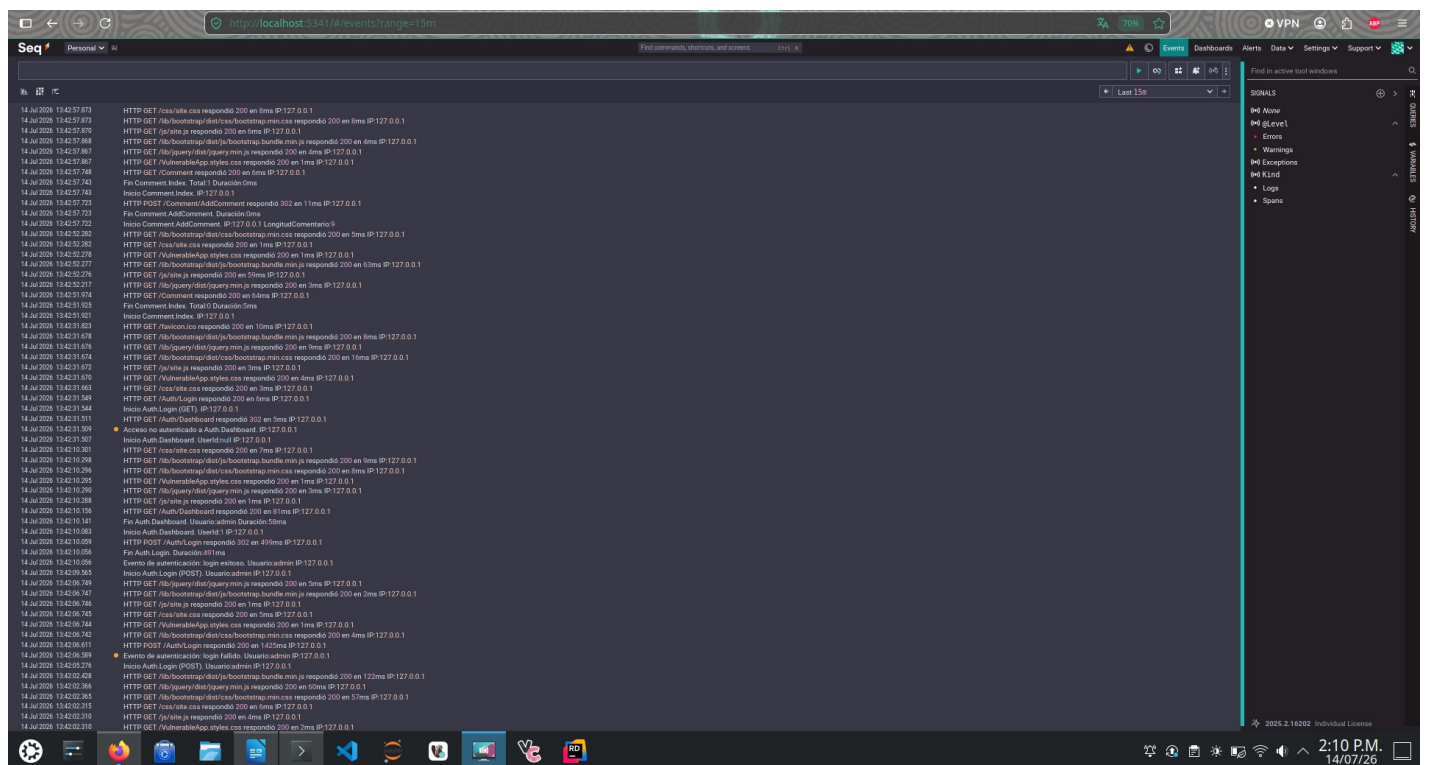
Configurar Promtail para recolectar y enriquecer los archivos de registro generados por VulnerableApp, enviarlos a Loki y validar que los mismos eventos puedan consultarse tanto desde Seq como desde Grafana, sin modificar la lógica de negocio de la aplicación.

Paso 1. Verificar la infraestructura existente

Se confirmó que los contenedores de Grafana, Loki y Promtail continúan en ejecución junto con Seq, y que VulnerableApp sigue generando registros en consola, archivo y Seq.

```
renyam@ellan-82x7i:~/Documentos/Two Cuatrimestre/Seguridad/VulnerableApp/observability$ docker ps
Orden «docker» no encontrada. Quizá quiso decir:
  la orden «docker» del paquete deb «podman-docker (5.7.0+ds2-3build1)»
  la orden «docker» del paquete deb «docker.io (29.1.3-0ubuntu4.1)»
Pruebe con: sudo apt install <nombre del paquete deb>
renyam@ellan-82x7i:~/Documentos/Two Cuatrimestre/Seguridad/VulnerableApp/observability$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                                NAMES
1f1868aea50    grafana/grafana:11.2.0             "/run.sh"               19 hours ago  Up 15 hours  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  grafana
cc716d318759    grafana/promtail:2.9.4             "/usr/bin/promtail -..." 19 hours ago  Up 28 minutes  0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp  promtail
7f86fd1d7093    grafana/loki:2.9.4                 "/usr/bin/loki -conf..." 19 hours ago  Up 15 hours  0.0.0.0:3100->3100/tcp, [::]:3100->3100/tcp  loki
91d3cf7f3dd    datast/seq:latest                  "/bin/seqentry"         7 days ago    Up 15 hours  443/tcp, 5341/tcp, 45341/tcp, 0.0.0.0:5341->80/tcp, 0.0.0.0:8081->80/tcp, [::]:5341->80/tcp, [::]:8081->80/tcp  seq
```

Evidencia: docker ps mostrando los 4 contenedores en ejecución (grafana, promtail, loki, seq).



Evidencia: Seq mostrando los eventos generados por VulnerableApp.

Paso 2. Analizar el archivo de configuración de Promtail

Antes de modificar el archivo, se analizó la función de cada sección del promtail-config.yml:

Sección	Función
server	Puerto HTTP/gRPC interno donde Promtail expone su propio estado y métricas; no está relacionado con el envío de logs a Loki.
positions	Ruta del archivo donde Promtail guarda el offset (última línea leída) de cada archivo monitoreado, para no reprocesar ni perder líneas al reiniciar.
clients	Endpoint de destino de los logs; en este caso la URL de push de Loki (http://loki:3100/loki/api/v1/push).
scrape_configs	Lista de "jobs" de recolección; cada uno define qué archivos leer, con qué labels y qué transformaciones aplicar (<code>pipeline_stages</code>).
static_configs	Dentro de un job, define el conjunto fijo de targets y labels base, a diferencia del descubrimiento dinámico de targets.
labels	Pares clave-valor adjuntos a cada línea enviada a Loki; constituyen la base de la indexación y de las consultas LogQL.

Paso 3. Configurar el monitoreo de múltiples archivos

Ruta utilizada dentro del contenedor de Promtail:

```
/var/log/vulnerableapp/log-*.txt
```

Correspondiente, vía bind mount, a la siguiente ruta en el host:

```
/home/renyam/Documentos/7mo Cuatrimestre/Seguridad/VulnerableApp/Logs/log-*.txt
```

VulnerableApp rota el archivo de log por día (log-20260709.txt, log-20260713.txt, log-20260714.txt, etc.). El patrón log-*.txt permite que Promtail descubra y monitoree automáticamente cada archivo nuevo que se genere, sin necesidad de editar la configuración cada día. Esto satisface el requisito de "monitorear más de un archivo de registro", en este caso con separación por fecha en lugar de por categoría.

VulnerableApp no genera un archivo security-*.txt separado: todos los eventos (peticiones HTTP, autenticación, operaciones CRUD) se centralizan en un único archivo por día. Por esta razón, la separación por categoría (Security, Audit, etc.) se logró mediante labels aplicadas por contenido de línea (`pipeline_stages`), y no por archivo de origen.

Promtail necesita conocer la ubicación física exacta de los archivos porque no "descubre" logs de forma automática: actúa como un lector en tiempo real (tail) de archivos reales en disco. Sin el `__path__` correcto, y sin el bind mount que expone la carpeta Logs del host dentro del contenedor, Promtail no tendría ningún archivo que leer.

Paso 4. Enriquecer los registros mediante labels

Se configuraron etiquetas adicionales para clasificar los eventos en dos dimensiones complementarias:

- `category`: Application (por defecto) / Security / Audit — clasifica el tipo de evento desde una perspectiva de negocio y seguridad.
- `module`: authentication / orders — identifica el componente técnico de la aplicación que originó el evento (CommentController se usó como equivalente de "orders", ya que la aplicación no cuenta con un controlador de órdenes real).
- `environment`: dev — identifica el entorno de despliegue.

Ventaja frente a la búsqueda textual: Loki indexa por labels, no por contenido completo de línea. Filtrar por {category="Security"} es una operación sobre el índice (rápida y económica), mientras que buscar texto libre implica escanear el contenido de cada línea. Combinar ambos enfoques (por ejemplo {category="Security"} |= "admin") es la forma eficiente de trabajar con LogQL.

Archivo promtail-config.yml actualizado:

```
server:
  http_listen_port: 9080
  grpc_listen_port: 0

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: vulnerableapp
    static_configs:
      - targets:
          - localhost
        labels:
          job: vulnerableapp
          app: VulnerableApp
          environment: dev
          category: Application
          __path__: /var/log/vulnerableapp/log-*.txt

    pipeline_stages:
      # category: Security -> fallos de autenticacion / accesos no autorizados
      - match:
          selector: '{job="vulnerableapp"}'
          stages:
            - regex:
                expression: '.*(?P<sec_hit>login fallido|Acceso no autenticado).*'
            - template:
                source: sec_hit
                template: '{{ if .Value }}Security{{ end }}'
            - labels:
                category: sec_hit

      # category: Audit -> acciones que cambian estado
      - match:
          selector: '{job="vulnerableapp"}'
          stages:
            - regex:
                expression: '.*(?P<audit_hit>login exitoso|logout|AddComment).*'
            - template:
                source: audit_hit
                template: '{{ if .Value }}Audit{{ end }}'
            - labels:
                category: audit_hit
```

```
# module: authentication
- match:
  selector: '{job="vulnerableapp"}'
  stages:
    - regex:
      expression: '.*(?P<auth_hit>Auth\\.|Auth/|autenticaci[oó]n).*'
    - template:
      source: auth_hit
      template: '{{ if .Value }}authentication{{ end }}'
    - labels:
      module: auth_hit

# module: orders (CommentController como equivalente)
- match:
  selector: '{job="vulnerableapp"}'
  stages:
    - regex:
      expression: '.*(?P<orders_hit>Comment\\.|Comment/).*'
    - template:
      source: orders_hit
      template: '{{ if .Value }}orders{{ end }}'
    - labels:
      module: orders_hit
```

```
Archivo Editar Ver Marcadores Complementos Preferencias Ayuda
~: sudo - Konsole
~: sudo x VulnerableApp: dotnet x
/home/roshan/Documents/Pent-Tool/nextjs/seguridad/vulnerableapp/observability/promtail-config.yml

# yaml file
#
# http_listen_port: 9090
# grpc_listen_port: 0

positions:
  filename: /tmp/positions.yml

clients:
  - url: http://localhost:9090

scrape_configs:
  - job_name: vulnerableapp
    static_configs:
      - targets:
          - localhost
    labels:
      job: vulnerableapp
      app: VulnerableApp
      namespace: dev
      category: Application
      url: /tmp/vulnerableapp/seg-*.txt
    pipeline_stages:
      # category: Security -> fallos de autenticación / acceso no autorizado
      - match:
          selector: '{job="vulnerableapp"}'
          stages:
            - regex:
                expression: '.*(Show_Autologs_Fallido|Acceso no autorizado).*'
            - template:
                source: sec_hit
                template: '{{ if .Value }}security{{ end }}'
            - labels:
                category: sec_hit

      # category: Audit -> acciones que cambian estado
      - match:
          selector: '{job="vulnerableapp"}'
          stages:
            - regex:
                expression: '.*(Show_Autologs_AccesoLogado|AddComment).*'
            - template:
                source: audit_hit
                template: '{{ if .Value }}audit{{ end }}'
            - labels:
                category: audit_hit

      # module: authentication
      - match:
          selector: '{job="vulnerableapp"}'
          stages:
            - regex:
                expression: '.*(Show_Autologs_Auth|autenticaci[oó]n).*'
            - template:
                source: auth_hit
                template: '{{ if .Value }}authentication{{ end }}'
            - labels:
                module: auth_hit

      # module: orders (CommentController como equivalente)
      - match:
          selector: '{job="vulnerableapp"}'
          stages:
            - regex:
                expression: '.*(Show_Autologs_AccesoLogado|Comment).*'
            - template:
                source: orders_hit
                template: '{{ if .Value }}orders{{ end }}'
            - labels:
                module: orders_hit
```

Evidencia: promtail-config.yml actualizado, abierto en el editor.

Paso 5. Aplicar la configuración

Se reinició únicamente el servicio Promtail para aplicar la nueva configuración:

```
docker compose restart promtail
docker compose logs promtail --tail 50
```

Los logs del contenedor confirmaron la recarga exitosa de la configuración ("Reloading configuration file") y la correcta asignación de las nuevas labels base (category="Application", environment="dev"), sin ningún mensaje de nivel error. Promtail retomó ambos archivos existentes desde el offset guardado en positions.yaml, sin reprocesar líneas ya enviadas.

Paso 6. Generar actividad en VulnerableApp

Se generó actividad variada en la aplicación para validar el pipeline completo:

- Autenticación fallida (usuario/contraseña incorrectos).
- Autenticación exitosa.
- Acceso no autenticado a una ruta protegida (/Auth/Dashboard).
- Operación CRUD: alta de un comentario nuevo (/Comment/AddComment).

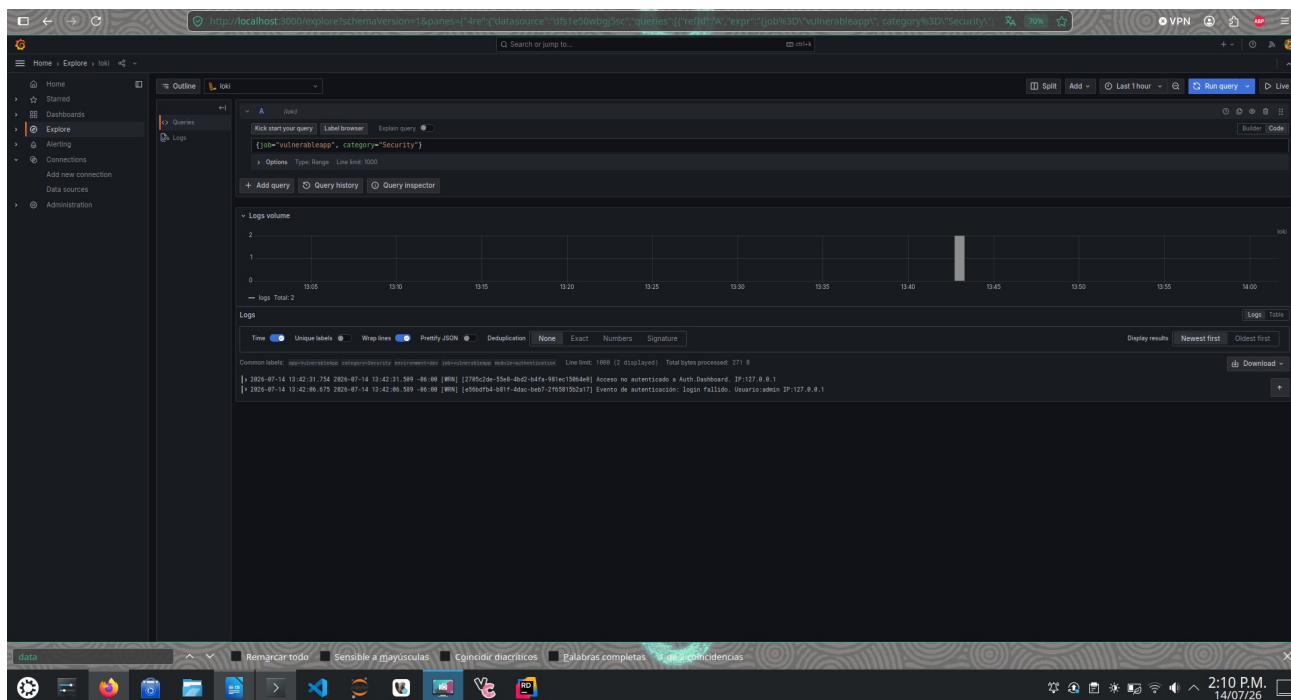
Se confirmó que todos estos eventos siguieron almacenándose correctamente en consola, en el archivo de log del día y en Seq antes de continuar con el análisis en Grafana.

Paso 7. Validar los registros desde Grafana

Se accedió a Explore, se seleccionó el datasource Loki y se realizaron las siguientes consultas LogQL utilizando las labels configuradas:

Consulta 1 — category="Security"

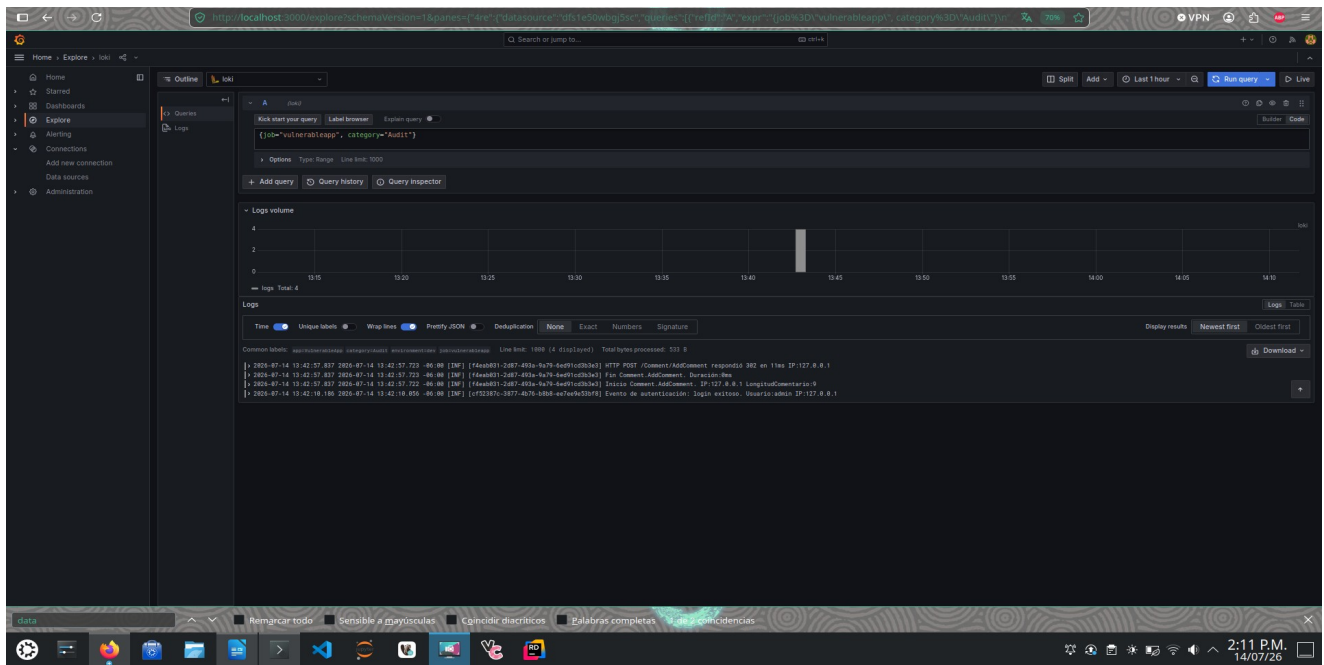
```
{job="vulnerableapp", category="Security"}
```



Resultado: 2 eventos (login fallido, acceso no autenticado).

Consulta 2 — category="Audit"

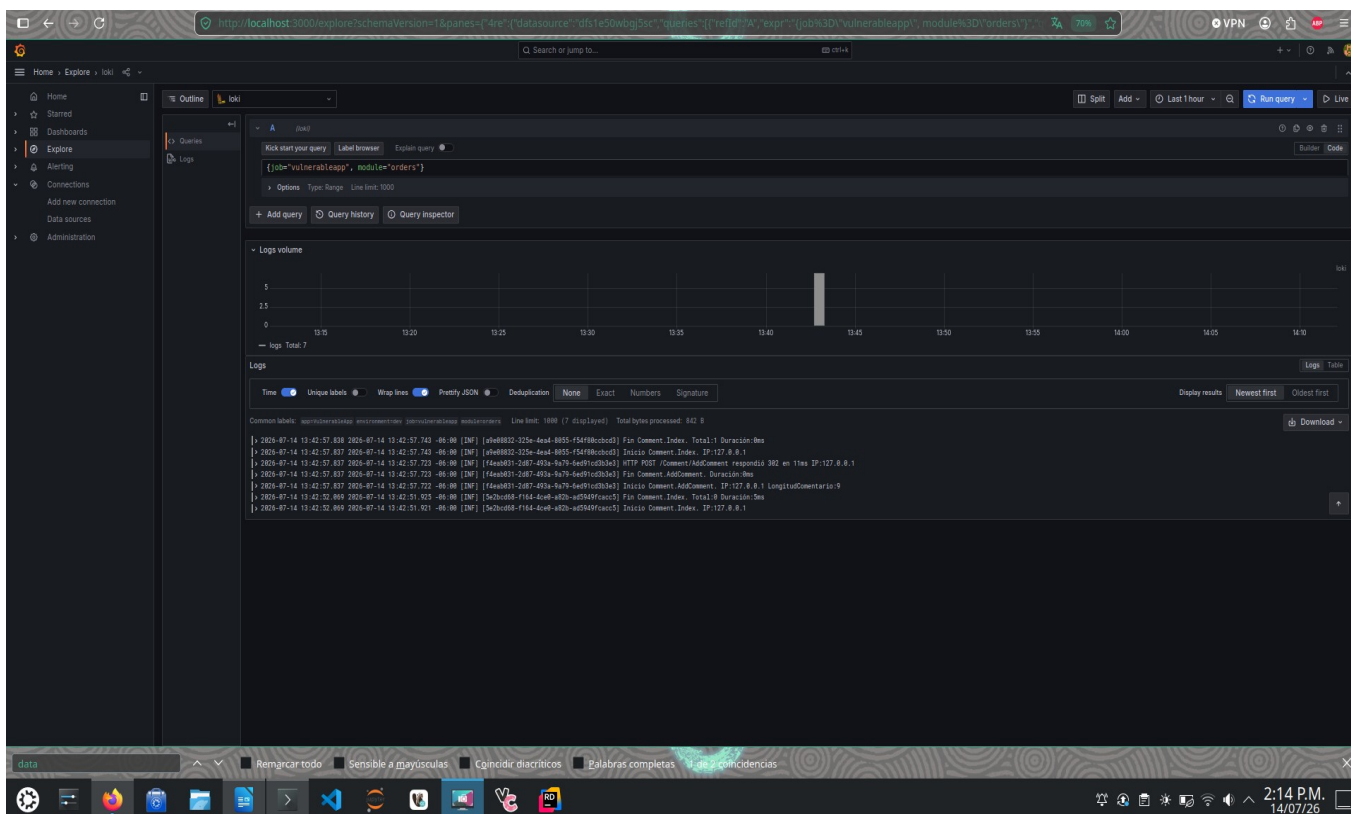
```
{job="vulnerableapp", category="Audit"}
```



Resultado: 4 eventos (login exitoso, alta de comentario).

Consulta 3 — module="orders"

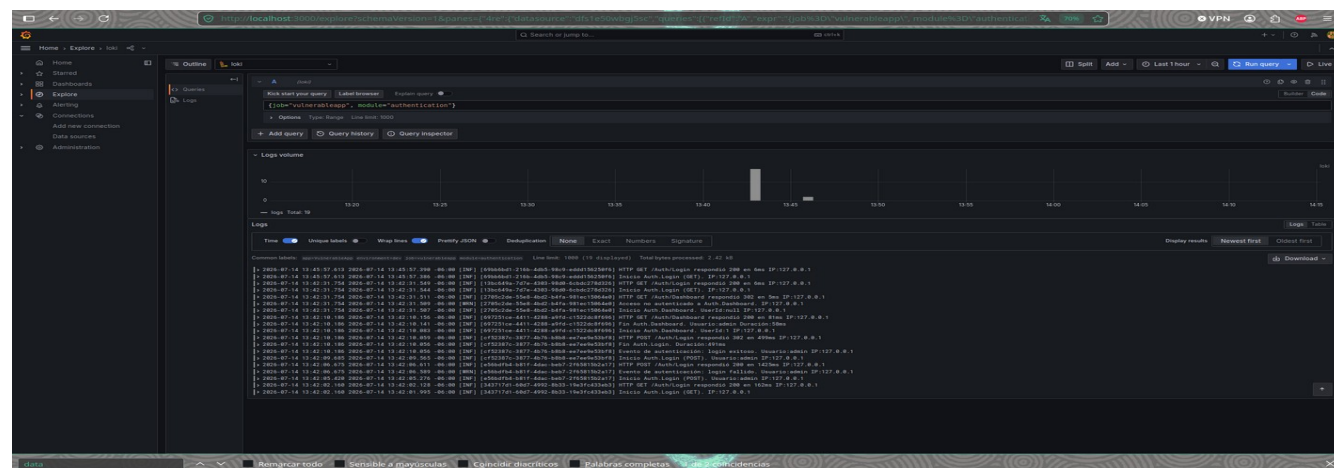
```
{job="vulnerableapp", module="orders"}
```

Resultado: 7 eventos relacionados con CommentController (Index y AddComment).

Consulta 4 — module="authentication" (Reto)

```
{job="vulnerableapp", module="authentication"}
```



Resultado: 19 eventos, todo el flujo del módulo de autenticación.

Paso 8. Comparar los resultados

Se analizaron los mismos eventos en Seq y en Grafana, confirmando coincidencia total (mismos timestamps y mismos mensajes).

Actividad de análisis

Característica	Seq	Grafana + Loki
Búsqueda textual	Sintaxis propia sobre propiedades estructuradas de Serilog; fuerte para filtrar por campos como Usuario, IP.	LogQL con <code> = "texto"</code> ; orientado a filtrar primero por stream (labels) y luego por contenido.
Filtros por labels	No existe el concepto de labels; se filtra por propiedades del evento estructurado.	Es el mecanismo central de indexación; cada combinación de labels define un stream independiente.
Dashboards	Limitados, orientados a inspección de eventos individuales.	Robustos y altamente personalizables, con múltiples tipos de visualización.
Consultas avanzadas	Consultas tipo SQL sobre propiedades del evento.	LogQL con funciones de agregación (<code>rate()</code> , <code>count_over_time()</code>), similar a PromQL.
Escalabilidad	Pensado para volúmenes moderados en desarrollo o entornos single-node.	Diseñado para producción a gran escala; separa índice de labels y almacenamiento de contenido.
Uso principal	Debugging fino de eventos individuales durante el desarrollo.	Observabilidad centralizada de múltiples fuentes en producción.

Pregunta de reflexión

¿Qué ocurriría si Promtail no almacenara la posición del último registro leído en `positions.yaml`?

Si Promtail no guardara la posición en `positions.yaml`, cada vez que el contenedor se reiniciara no sabría desde dónde continuar leyendo cada archivo. Esto genera dos escenarios problemáticos: reenviar todo el archivo desde el inicio (duplicando en Loki los eventos ya procesados), o comenzar a leer solo desde el final del archivo (perdiendo cualquier línea nueva escrita mientras el contenedor estaba detenido). En ambos casos se rompe la garantía de una ingesta confiable de logs, generando duplicados o huecos en la observabilidad del sistema.

Reto

Se configuraron las labels adicionales `environment=dev`, `module=authentication` y `module=orders` (ver Paso 4 y archivo de configuración completo arriba). Posteriormente se utilizó Grafana para filtrar únicamente los registros del módulo Authentication mediante la consulta:

```
{job="vulnerableapp", module="authentication"}
```

El resultado (Consulta 4, Paso 7) mostró 19 eventos correspondientes exclusivamente a las acciones del controlador de autenticación (inicios de sesión exitosos y fallidos, accesos no autenticados, y consultas al dashboard), confirmando que la clasificación por label funciona correctamente de forma independiente a la clasificación por category.

Conclusiones del análisis comparativo

A lo largo de esta práctica se confirmó que Seq y Grafana+Loki, aunque cubren el mismo flujo de datos (Serilog → archivo → observabilidad), tienen propósitos complementarios. Seq destaca en la inspección detallada de eventos individuales durante el desarrollo, gracias a su búsqueda sobre propiedades estructuradas del log. Grafana+Loki, en cambio, demostró su fortaleza en la clasificación y filtrado masivo mediante labels (category, module, environment), permitiendo aislar en segundos subconjuntos de eventos —por ejemplo, los 19 eventos del módulo de autenticación— sin necesidad de leer línea por línea.

Esta capacidad de indexar por label, en vez de solo por texto, es lo que hace a Loki más adecuado para escalar en entornos de producción con grandes volúmenes de logs, mientras que Seq resulta más ágil para debugging puntual en desarrollo local. Ambas herramientas, integradas sin modificar la lógica de negocio de VulnerableApp, demuestran que es posible evolucionar la observabilidad de un sistema de forma incremental y desacoplada de su código fuente.